Introduction to Applied Machine Learning

04: Artificial Neural Networks and the Multilayer Perceptron

🎓 Machine Learning Adventure 🚀

1. 🤖 Introduction to Machine Learning
2. 🧠 Artificial Neural Networks and Computer Vision
3. 🔤 Transformers and Language Models
4. 🎨 GANs and Diffusion for Image Generation
5. 🎮 Reinforcement Learning and Behavior
6. 🏆 Final Projects

Today's Learning Objectives

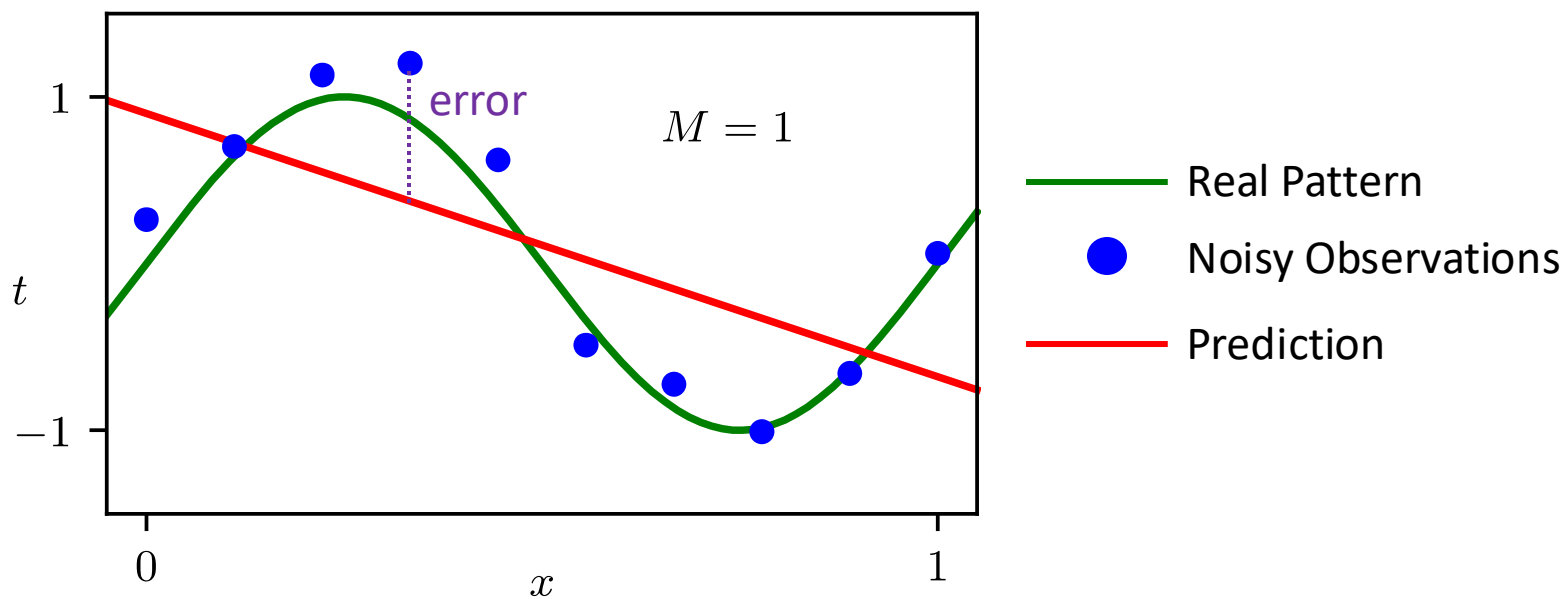
1. Nonlinear representations and how to learn them
2. Alternative approaches to neural networks and deep learning
3. The Multilayer Perceptron Neural Network Architecture
4. Forward Propagation and Inference with a Neural Network

Next time: Backpropagation and Minibatch Stochastic Gradient Descent for **training** neural networks.

Concrete Example: Learning from Data (adopted from BB1)

Suppose we have a number of data points, and we want to estimate the underlying pattern that matches the data.

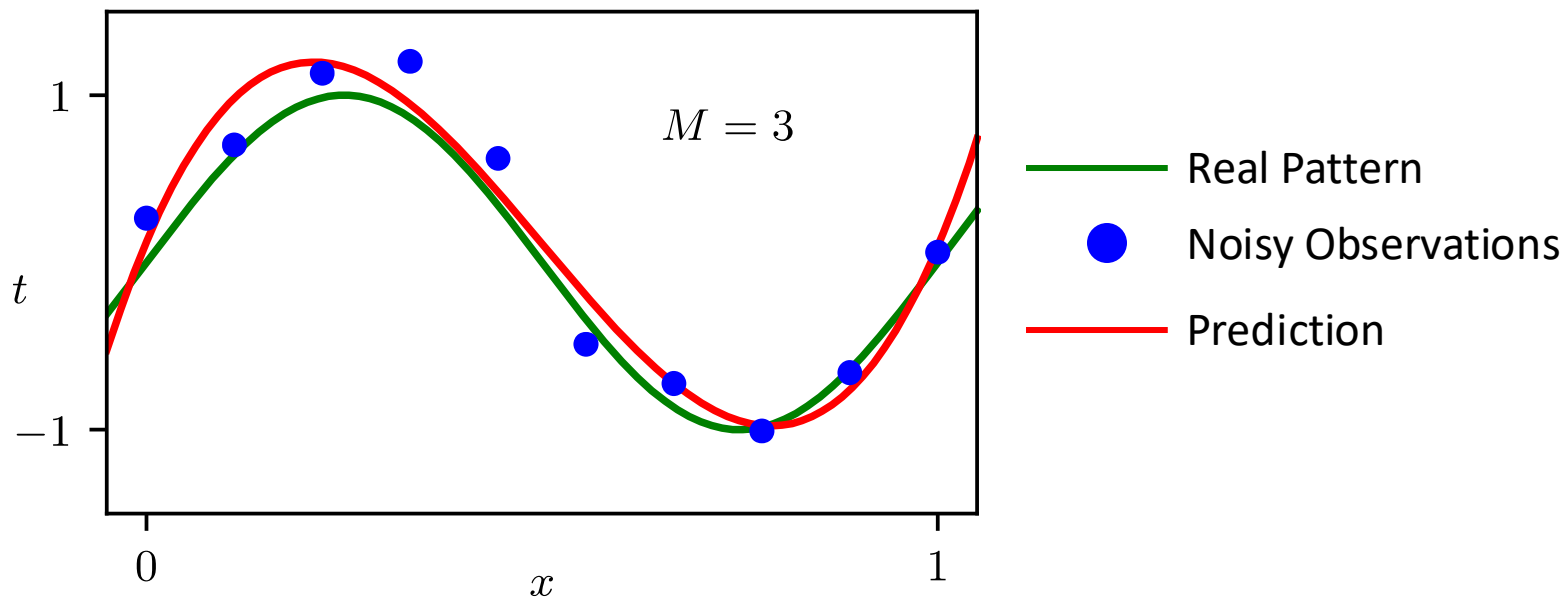
We could speculate that t is a simple ***linear function*** of x (the linear weights are called model ***parameters***) and try to draw the line that minimizes the average error.



Concrete Example: Learning from Data (adopted from BB1)

Suppose we have a number of data points, and we want to estimate the underlying pattern that matches the data.

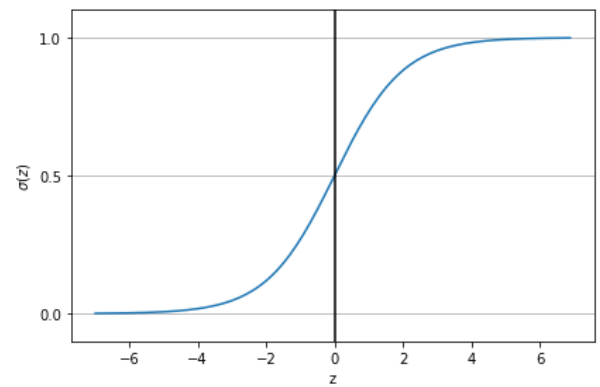
To improve our ***training error***, we might use a more complex model, such as a cubic function of the input.

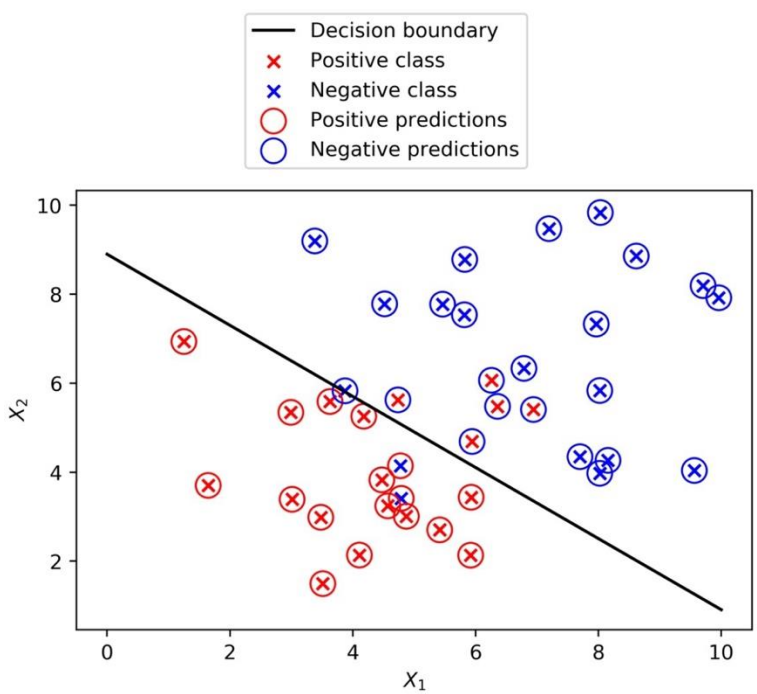


Geometric Interpretation: Linear Decision Boundary

- Predict $\hat{y} = 1$ if $p > 0.5$ (default), else $\hat{y} = 0$
- Implies a **decision boundary** at

$$\sigma\left(b + \boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{x}\right) = 0.5$$
$$\rightarrow b + \boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{x} = 0$$



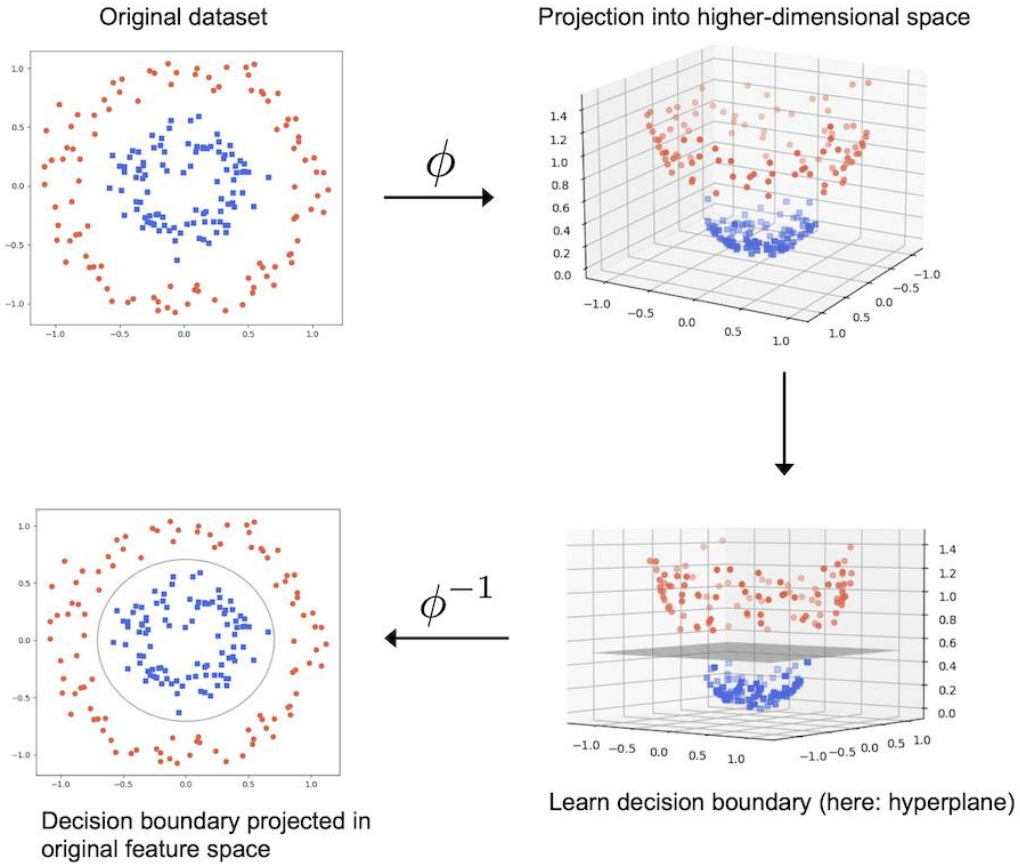


The model has:

- Low confidence ($p \approx 0.5$) for points **near** the boundary
- High confidence ($p \approx 0$ or 1) for points far from the boundary

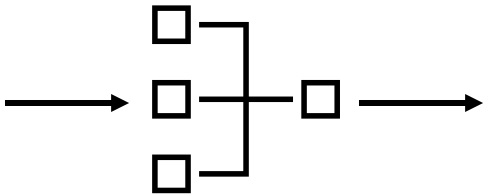
Nonlinear Representations

Like linear regression with polynomial features: Represent data in a higher dimensional space and learn a linear boundary in that transformed space.



Real World Example

In many complex applied ML tasks, linear models of raw data don't suffice. For example, FaceID:



90% probability "true" is your face

```
[[170 172 163 161 171 181 184 177]
 [162 161 155 150 163 173 174 169]
 [148 154 147 155 158 153 164 167]
 [ 98 130 135 160 152 139 156 167]
 [ 50  92 145 165 151 142 154 166]
 [ 43  92 161 157 141 137 152 162]
 [ 69 134 161 147 137 126 150 159]
 [101 157 148 144 127 102 143 157]]
```

A linear model of raw pixels is too simple: A given pixel being brighter does not, by itself, make it more or less likely to be your face.

Choosing the right representation? Manual Feature Engineering

You pre-compute features you think might be relevant. For example: Suppose we are predicting house prices.

- Given: address, size, year, bedrooms, bathrooms
- Compute additional features: price-per-sq-ft in neighborhood, distance to elementary school, …

Pros

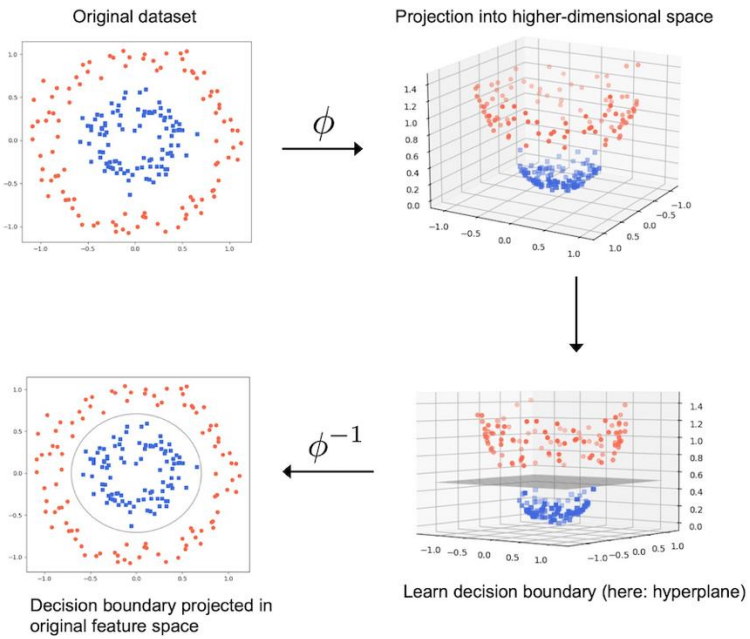
- Interpretable
- Can use simple models of complex features
- Works with limited data

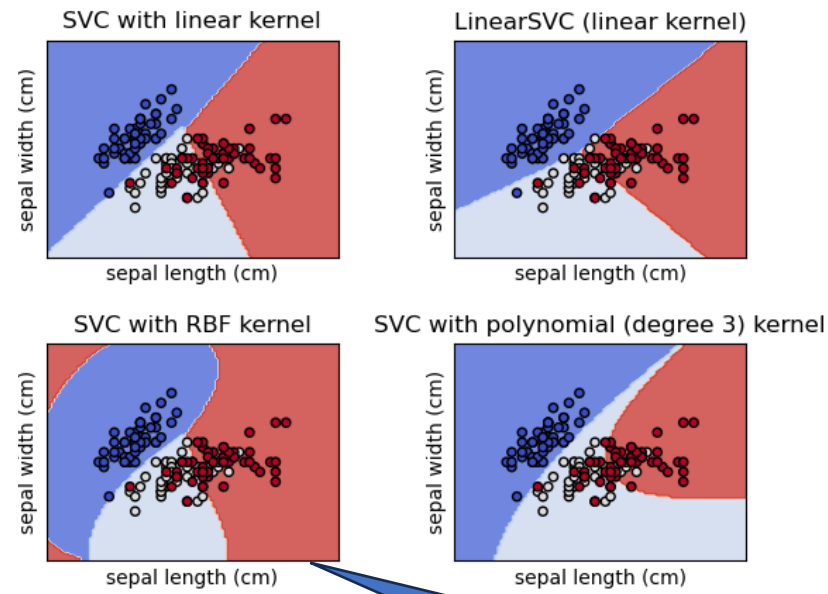
Cons

- Labor intensive for every task and dataset
- Need domain expertise
- Hard to scale beyond simple tabular data

How to Learn a Representation? Support Vector Machine (SVM)

Represent data using *distances* between pairs of points to *implicitly* represent in a high dimensional space (kernel trick).



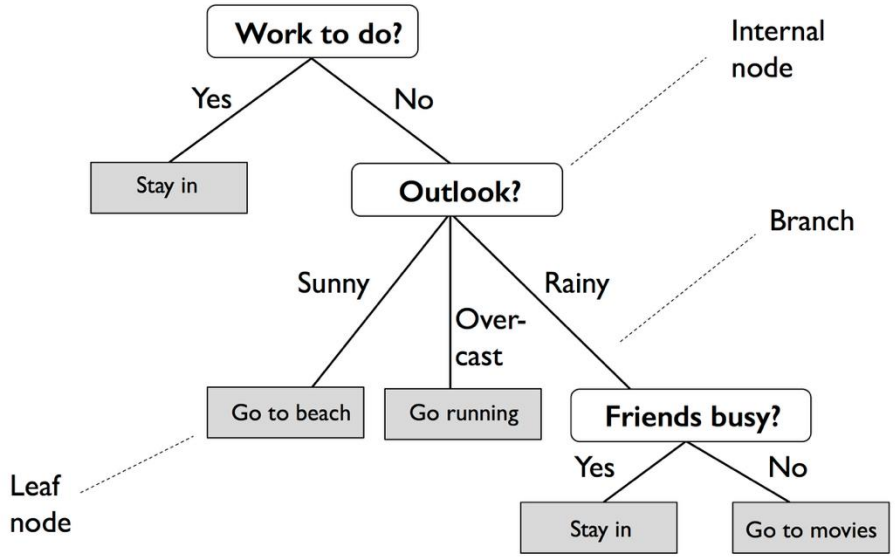


For m data points and n features, avoids the $O(n^d\, m)$ complexity of explicitly computing a degree d polynomial representation – instead $O(m^2\, n)$ to compute distances.

Radial Basis Function (RBF): Computes a similarity between points that decreases exponentially with distance.

How to Learn a Representation? Decision Trees

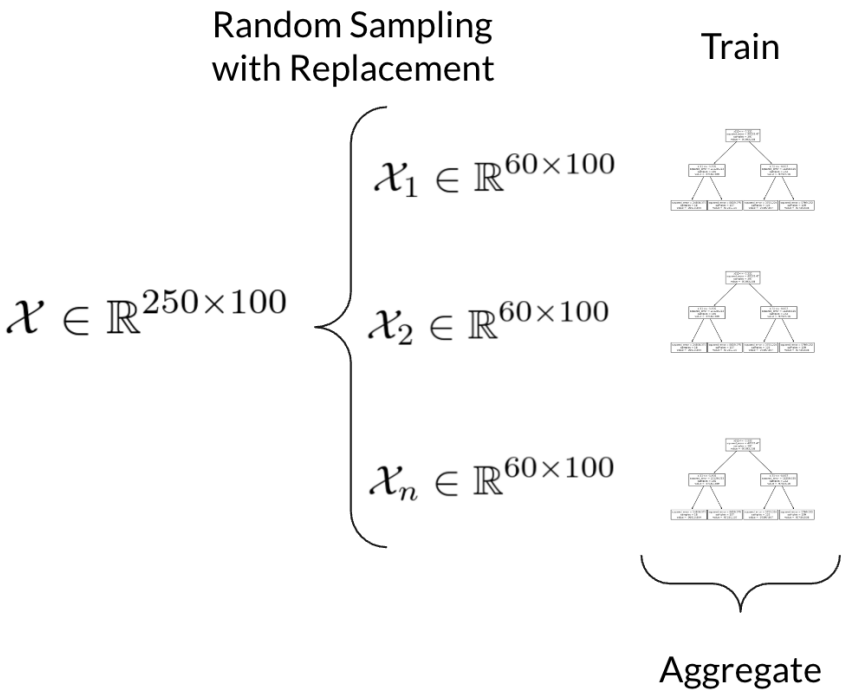
Decide a feature, decide a split on that feature, recurse.
To predict: follow to a leaf node and predict the mode (classification) or mean (regression) of the training data.



Represents data as a vector of Boolean values for conditionals over the original feature space.

How to Learn a Representation? Ensemble Methods

For example, a **Random Forest.** Randomly sample several subsets of the data and train a decision tree for each subset.



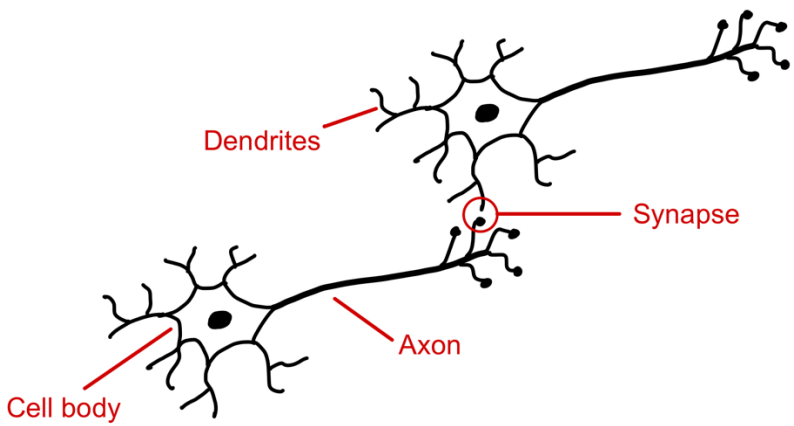
An example of what is called **bagging** for combining many **weak** learners into one stronger model (while avoiding overfitting) by training on different subsets of data.

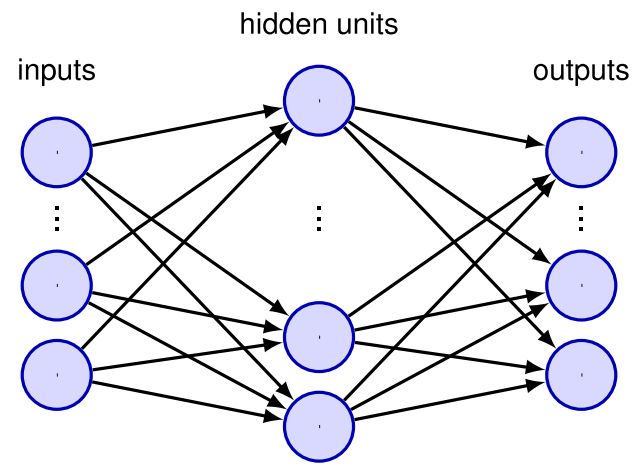
Predict by letting the trees "vote," e.g., modal prediction, or average probability estimates. Data represented as a distribution over decision tree encodings.

Figure credit: By Harry585 - Own work, CC BY-SA 4.0, source

How to Learn a Representation? Neural Networks

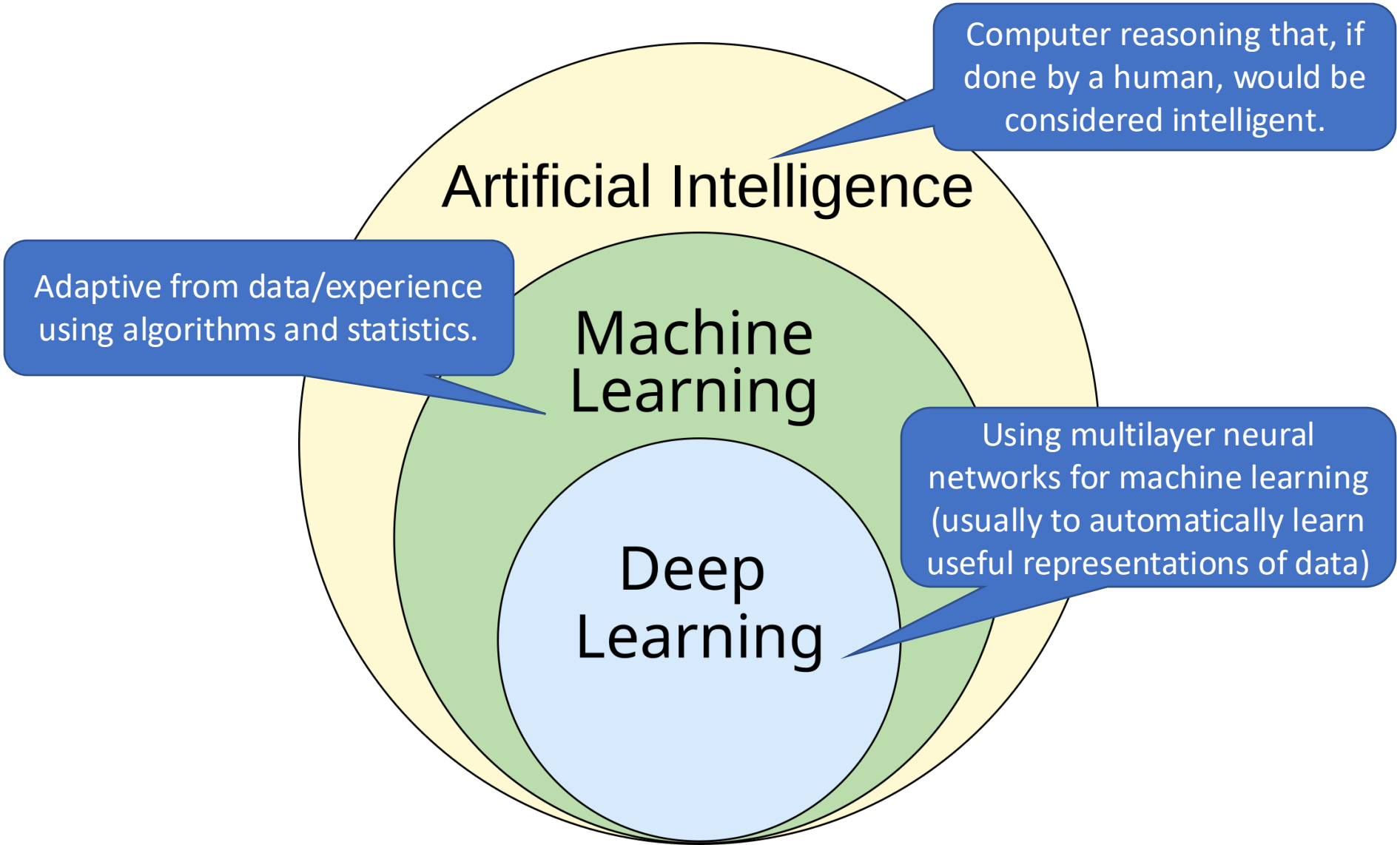
Specify a computational graph, called an ***artificial neural networks*** (***deep learning***).





Nonlinear representation of inputs in the “Hidden Units” – a representation that is learned automatically from data.

Characterizing the Fields

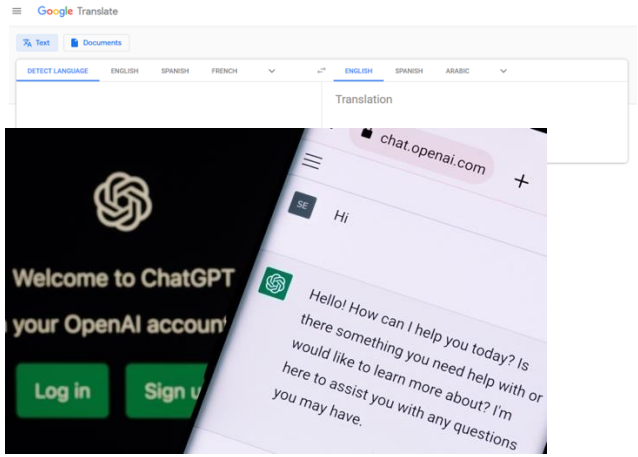


The Deep Learning Revolution

Since the ~2012 “Deep Learning Revolution,” Artificial Neural Networks have become the most widely used models for applications on large data and complex tasks.

Computer Vision

Natural Language Processing



Science

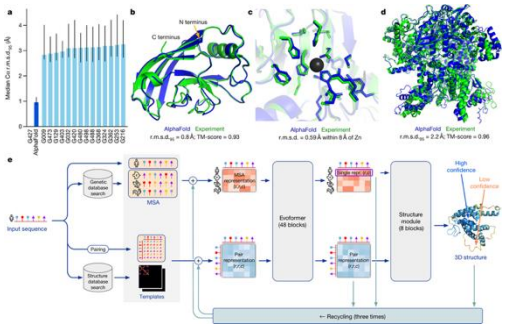


Image Generation

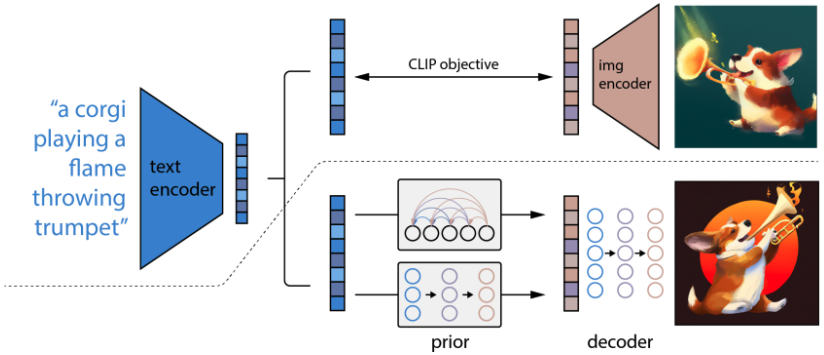


Figure 2: A high-level overview of unCLIP. Above the dotted line, we depict the CLIP training process, through which we learn a joint representation space for text and images. Below the dotted line, we depict our text-to-image generation process: a CLIP text embedding is first fed to an autoregressive or diffusion prior to produce an image embedding, and then this embedding is used to condition a diffusion decoder which produces a final image. Note that the CLIP model is frozen during training of the prior and decoder.

And more!

Mathematical Foundations: Universal Function Approximation

Home > Mathematics of Control, Signals and Systems > Article

Approximation by superpositions of a sigmoidal function

Published: December 1989

Volume 2, pages 303–314, (1989) Cite this article

G. Cybenko

Abstract

In this paper we demonstrate that finite linear combinations of compositions of a fixed, univariate function and a set of affine functionals can uniformly approximate any continuous function of n real variables with support in the unit hypercube; only mild conditions are imposed on the univariate function. Our results settle an open question about representability in the class of single hidden layer neural networks. In particular, we show that arbitrary decision regions can be arbitrarily well approximated by continuous feedforward neural networks with only a single internal, hidden layer and any continuous sigmoidal nonlinearity. The paper discusses approximation properties of other possible types of nonlinearities that might be implemented by artificial neural networks.

Leshno et al. '93 extended to other activations

When NOT to Use Deep Learning

We will spend most of the rest of the course on deep learning

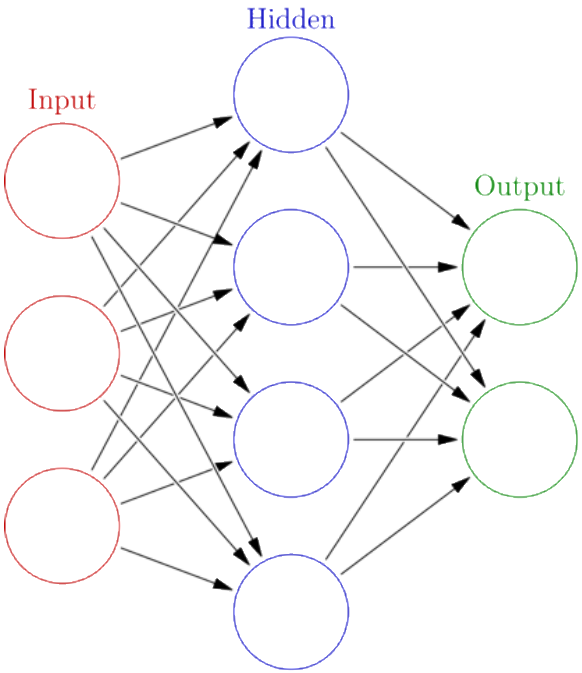
- Empirically, state-of-the-art (SOTA) results across diverse applied problems in
  - Computer Vision,
  - Natural Language Processing,
  - Generative modeling,
  - Reinforcement learning,
  - …
- Industry standard for complex, high-dimensional data

But you don't need deep learning for everything

- Little data: (< 1,000s) Neural Networks overfit easily
- Few features: (simple tabular data) Little to no advantage to deep learning – just use a simpler model
- Interpretability: Linear models and decision trees are easier to understand and explain

Basic Architecture of a Feedforward Neural Network

Input layer is a vector of length equal to your input data.



Output layer is a vector of length equal 1 (regression or binary classification) or k (for classification with k categories).

Model *parameters* are linear coefficients and bias terms.

Nonlinear *activation functions* are used for the hidden units.

Hidden layer(s) hold intermediate feature representations, you choose the size (more hidden units = greater model capacity).

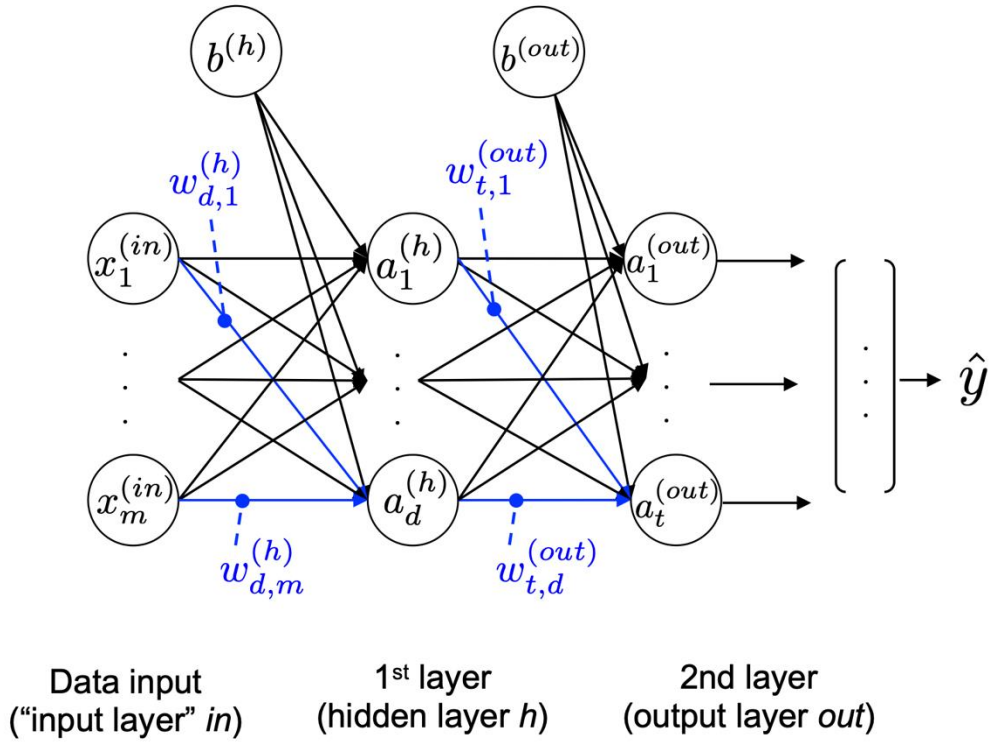
Multilayer Perceptron and Inference: Forward Propagation

$$\boldsymbol{a}^{(h)} = W^{(h)}\,\boldsymbol{x}^{(in)} + b^{(h)}$$

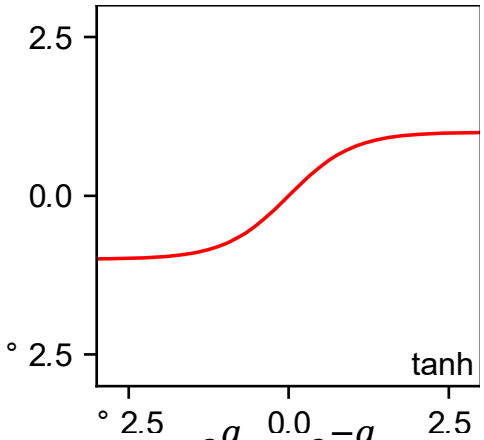
$$\boldsymbol{a}^{(out)} = W^{(out)}\,\sigma\left(\boldsymbol{a}^{(h)}\right) + b^{(out)}$$

Multiplies weight matrix by inputs, add bias, then element-wise activation.

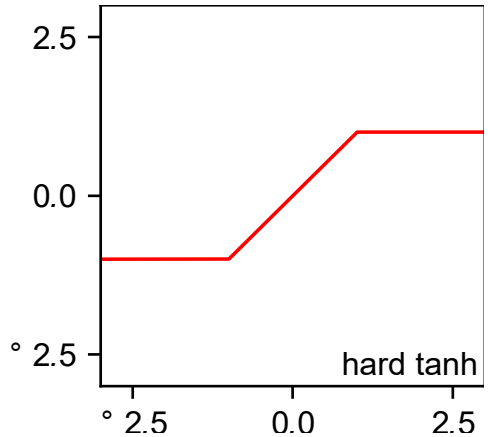
- Linear combination of weights and features
- Nonlinear activation function
- Possible output normalization (e.g., softmax)



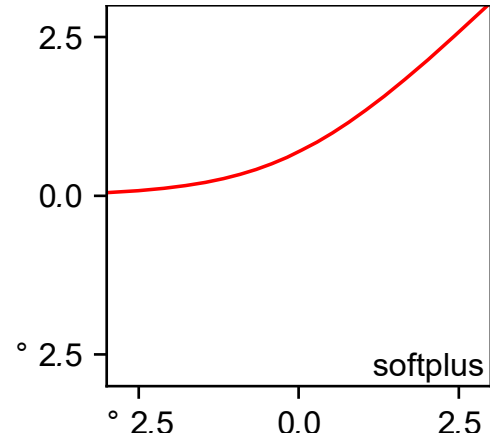
Activation Functions (beyond sigmoid)



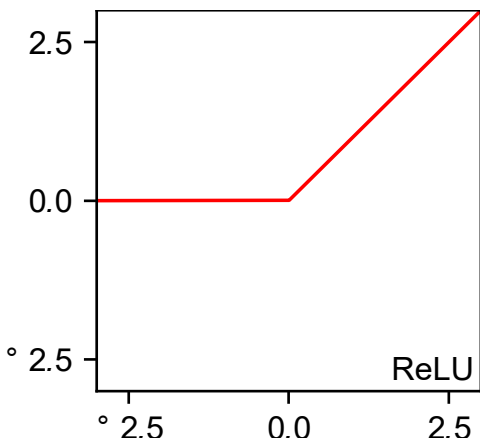
$$\frac{e^a - e^{-a}}{e^a + e^{-a}}$$



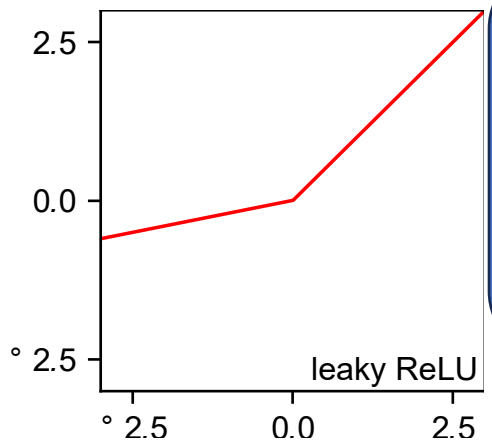
$$\max(-1, \min(1, a))$$



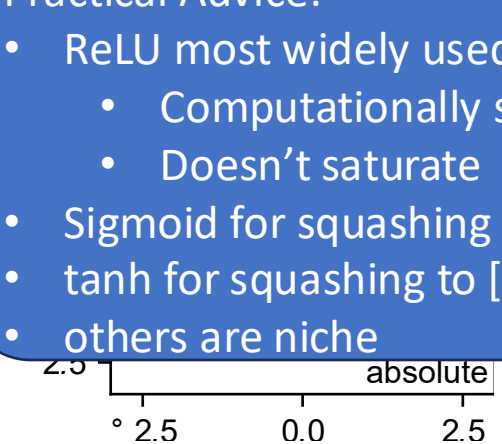
$$\ln(1 + e^a)$$



$$\max(0, a)$$



$$\max(0, a) + \alpha \min(0, a)$$



$$|a|$$

Practical Advice:

- ReLU most widely used
  - Computationally simple
  - Doesn't saturate
- Sigmoid for squashing to [0, 1]
- tanh for squashing to [-1, 1]
- others are niche

Common Output Units and Loss/Error Functions

For a regression task:	**No/linear output activations**, just final linear layer with **mean squared error**
For a binary classification task:	**Sigmoid** output activation for conditional probability estimation, **binary cross entropy** loss, **threshold** to classify
For a multiclassification task:	**Softmax** output activation for conditional probability estimation, **cross entropy** loss, **max probability** to classify

Multilayer Perceptron and Inference: Forward Propagation

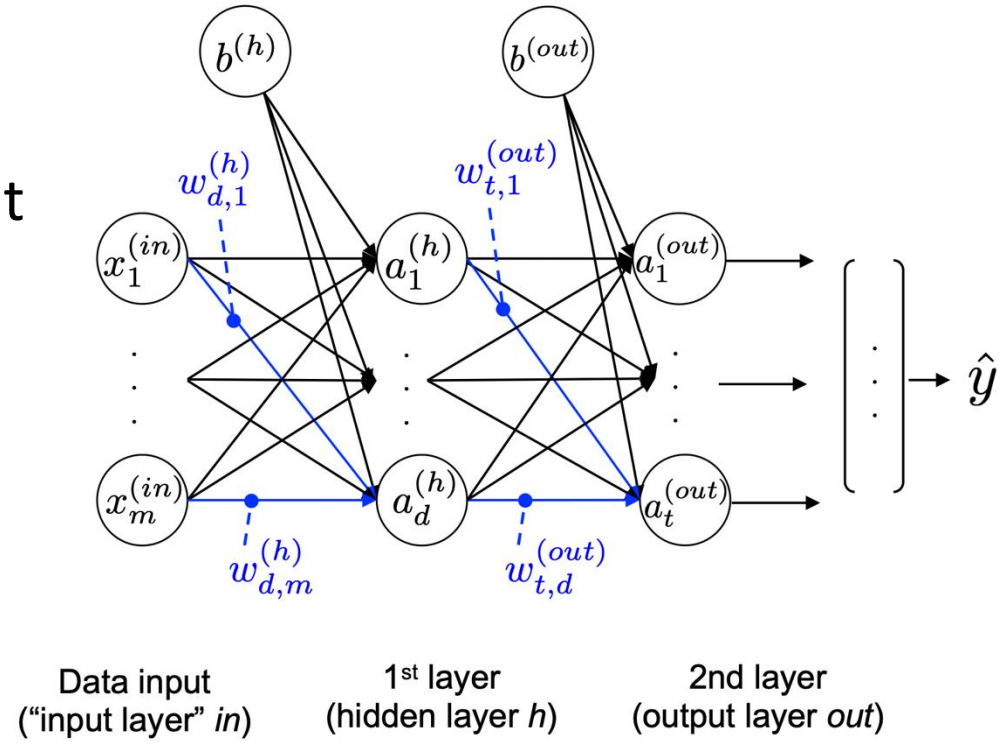
$$\boldsymbol{a}^{(h)} = W^{(h)}\,\boldsymbol{x}^{(in)} + b^{(h)}$$

$$\boldsymbol{a}^{(out)} = W^{(out)}\,\sigma\left(\boldsymbol{a}^{(h)}\right) + b^{(out)}$$

Multiplies weight matrix by inputs, add bias, then element-wise activation.

Runtime complexity for inference with forward propagation for a single input data point is ***linear*** with respect to the total number of ***parameters***.

(Runtime dominated by the matrix multiplications)



Multilayer Perceptron How Many Parameters?

$$\boldsymbol{a}^{(h)} = W^{(h)}\,\boldsymbol{x}^{(in)} + b^{(h)}$$

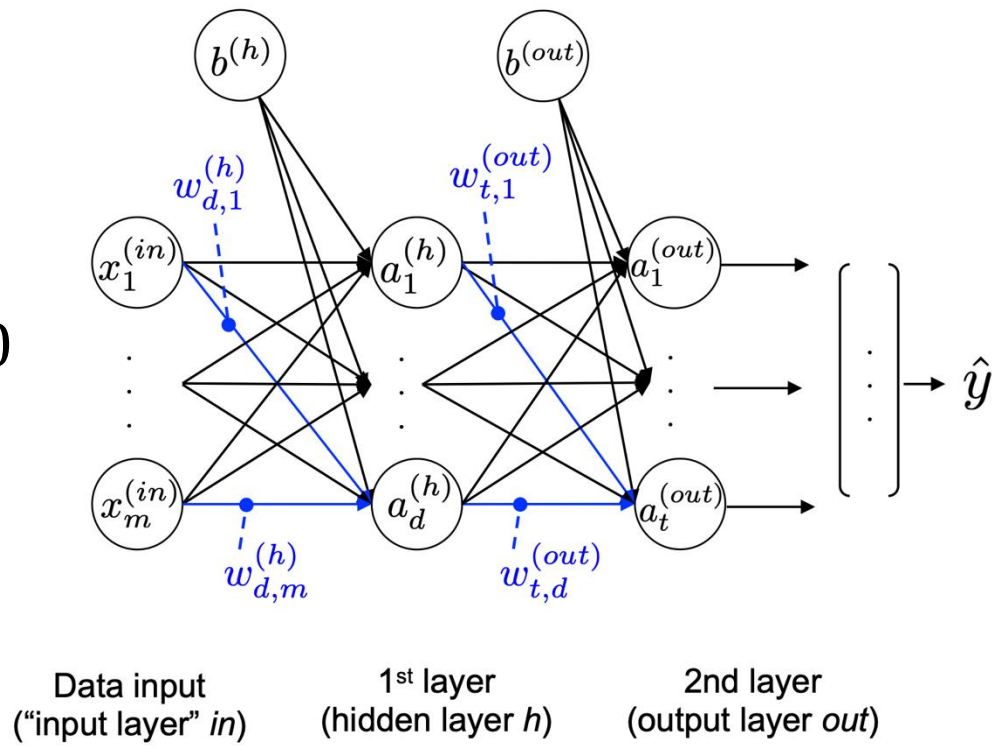
$$\boldsymbol{a}^{(out)} = W^{(out)}\,\sigma\left(\boldsymbol{a}^{(h)}\right) + b^{(out)}$$

Multiplies weight matrix by inputs, add bias, then element-wise activation.

A linear layer with m input units and d hidden units has $(m+1)d$ parameters.

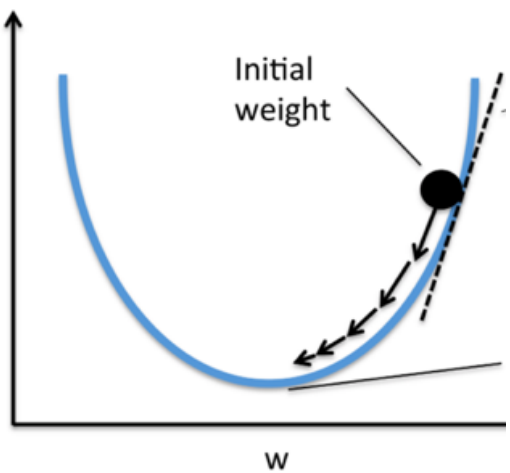
MLP with 60 input units, 100 hidden units, and 2 output units has up to:

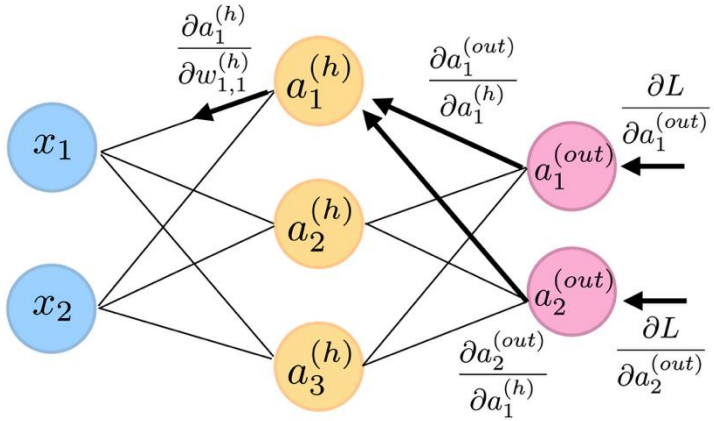
$(61)(100) + (101)(2) =$ 6,302 parameters.



Training: How to Learn Good Parameters?

- The short answer is…
  - Minibatch Stochastic Gradient Descent
  - With Backpropagation to Calculate Derivatives
- Which we will study **next time**.





Gradient for hidden layer weight:

$$\frac{\partial L}{\partial w_{1,1}^{(h)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}} + \frac{\partial L}{\partial a_2^{(out)}} \cdot \frac{\partial a_2^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}}$$

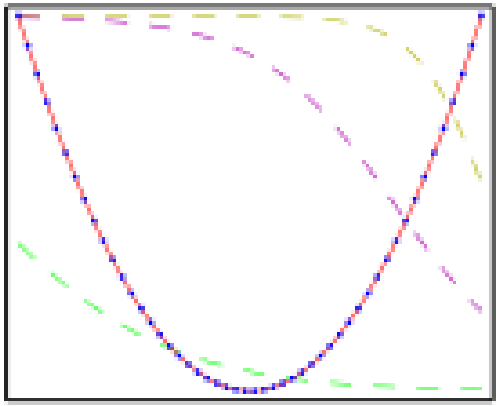
Small Function Approximation Example

This example visualizes how a 2-layer network with a single input unit, 3 hidden units with tanh activations, and a single output unit with no/linear output activation, can represent diverse functions.

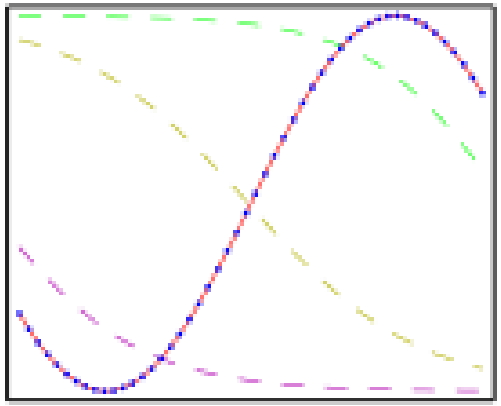
Data points

Learned network output

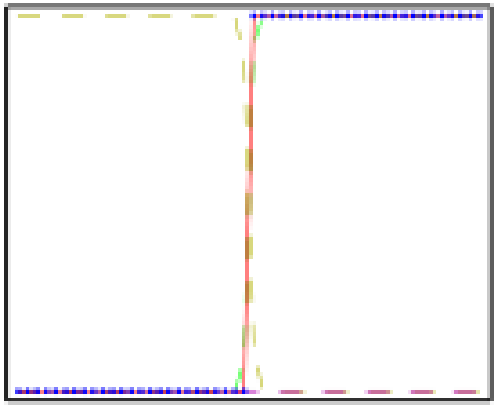
Hidden unit outputs



$f(x) = x^2$



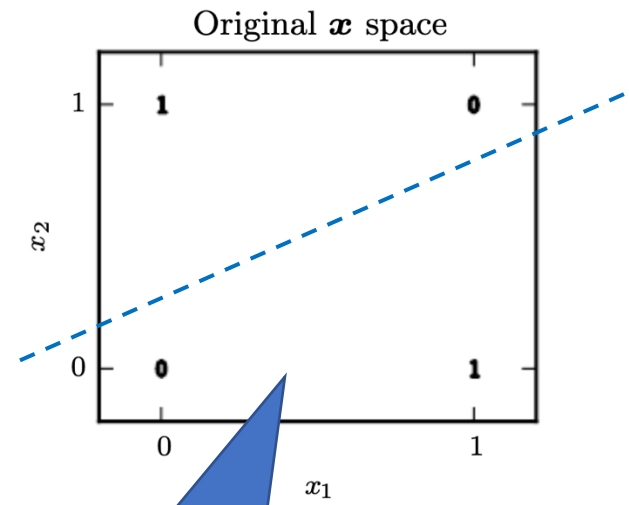
$f(x) = \sin(x)$



$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Example: Solving XOR

Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

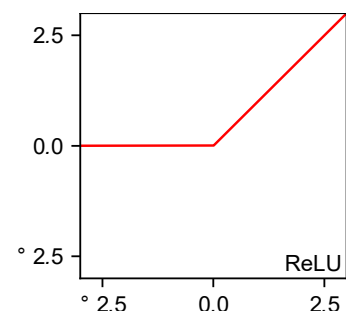


Linear decision boundary does not classify all correctly

Let's specify an MLP with:

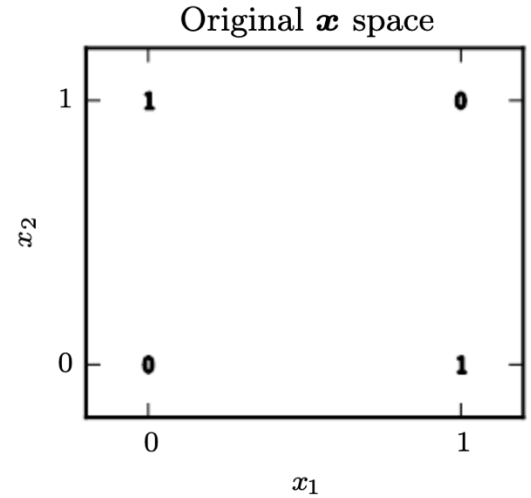
- 2 hidden units
- A bias vector for the hidden units
- ReLU activation for the hidden units
- Single output unit
- Simple linear output layer

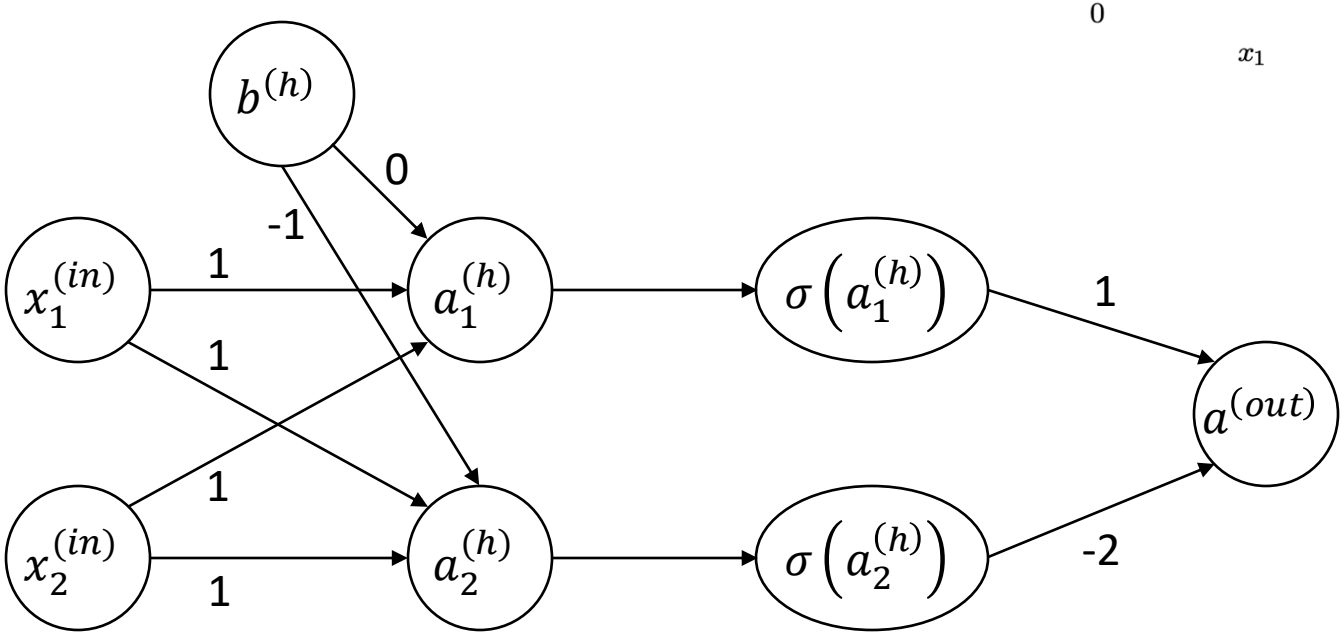
$$\mathrm{ReLU}(z) = \max(0, z)$$



Example: Solving XOR

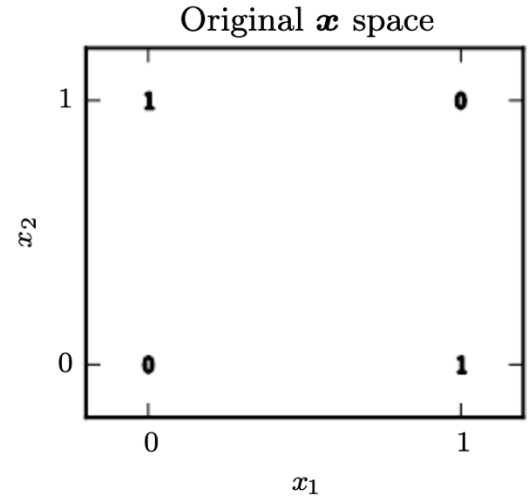
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

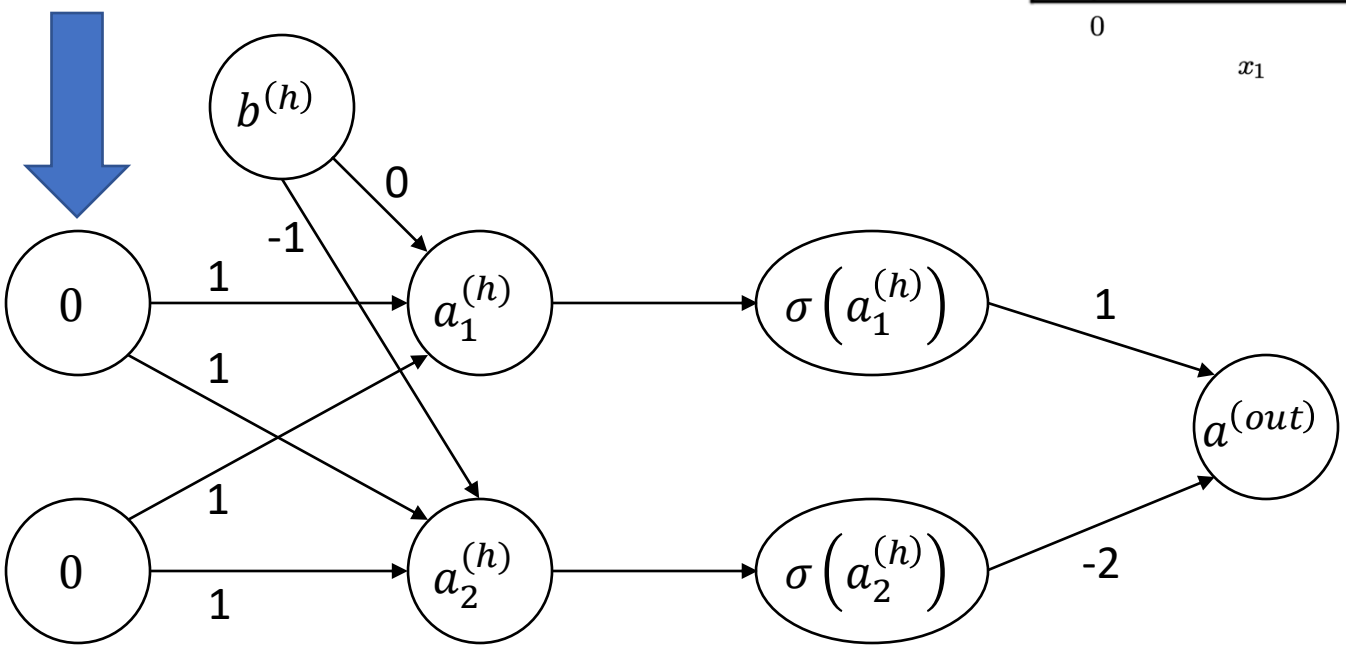




Example: Solving XOR

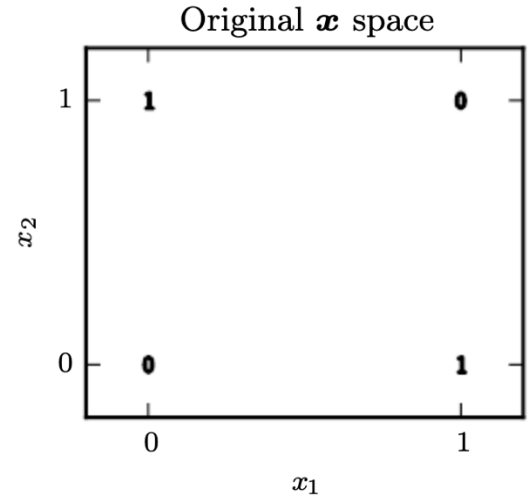
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

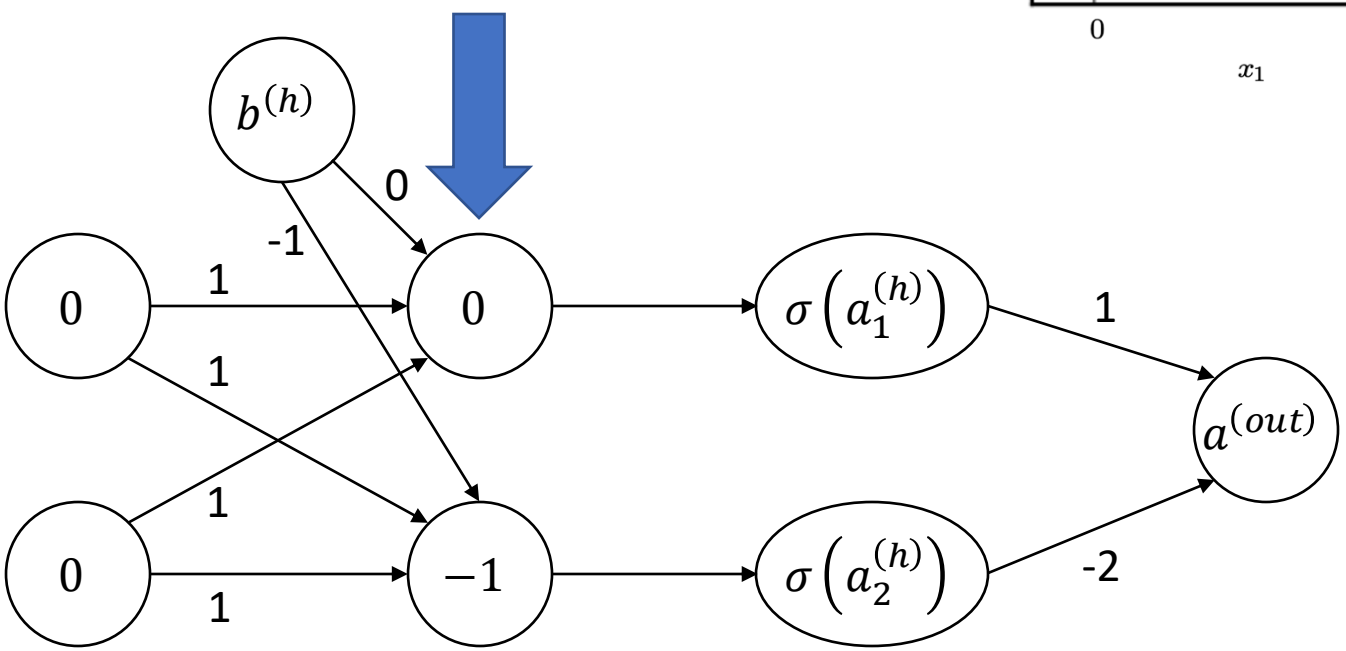




Example: Solving XOR

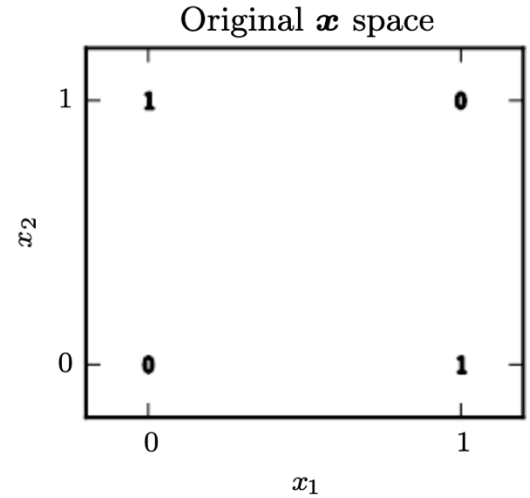
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

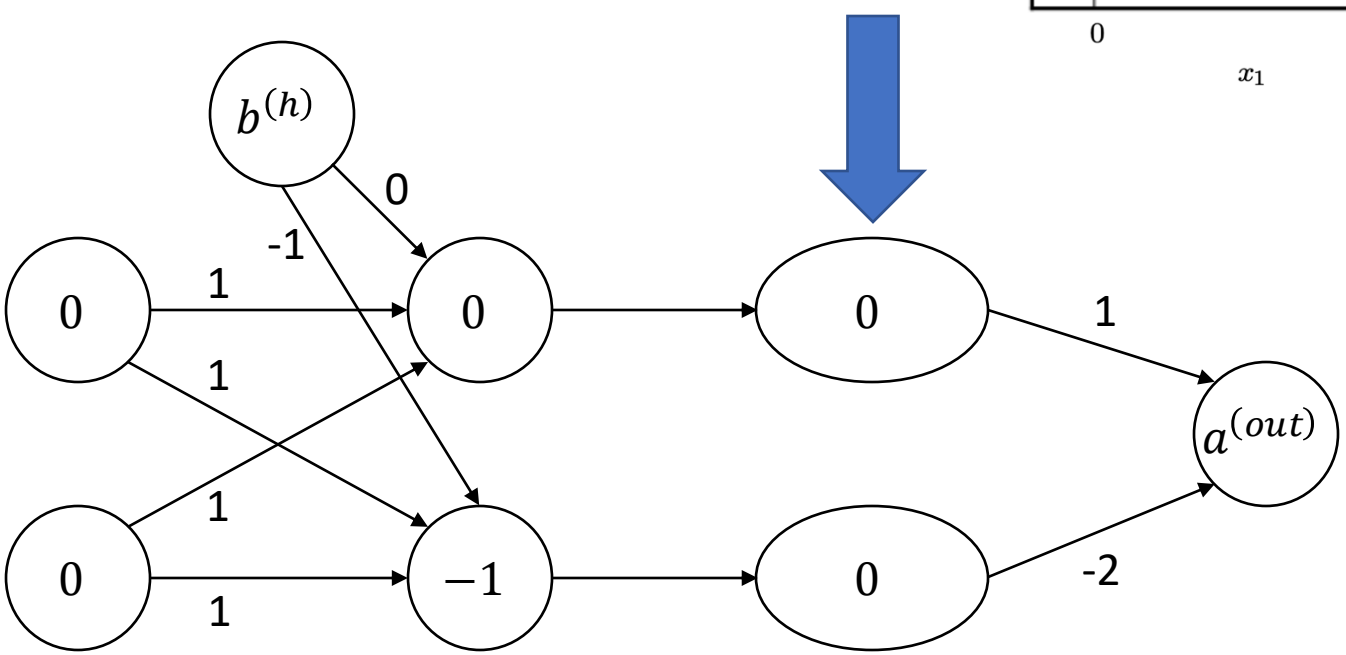




Example: Solving XOR

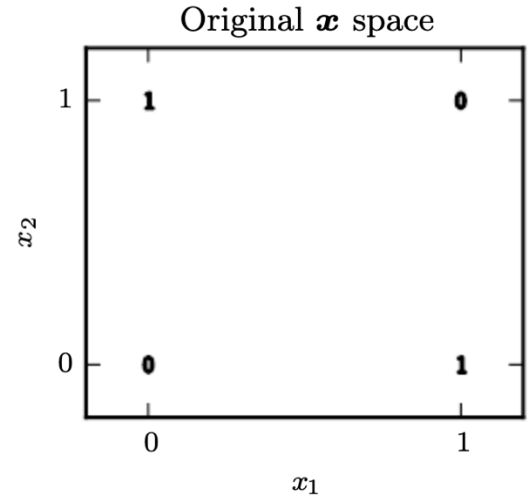
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

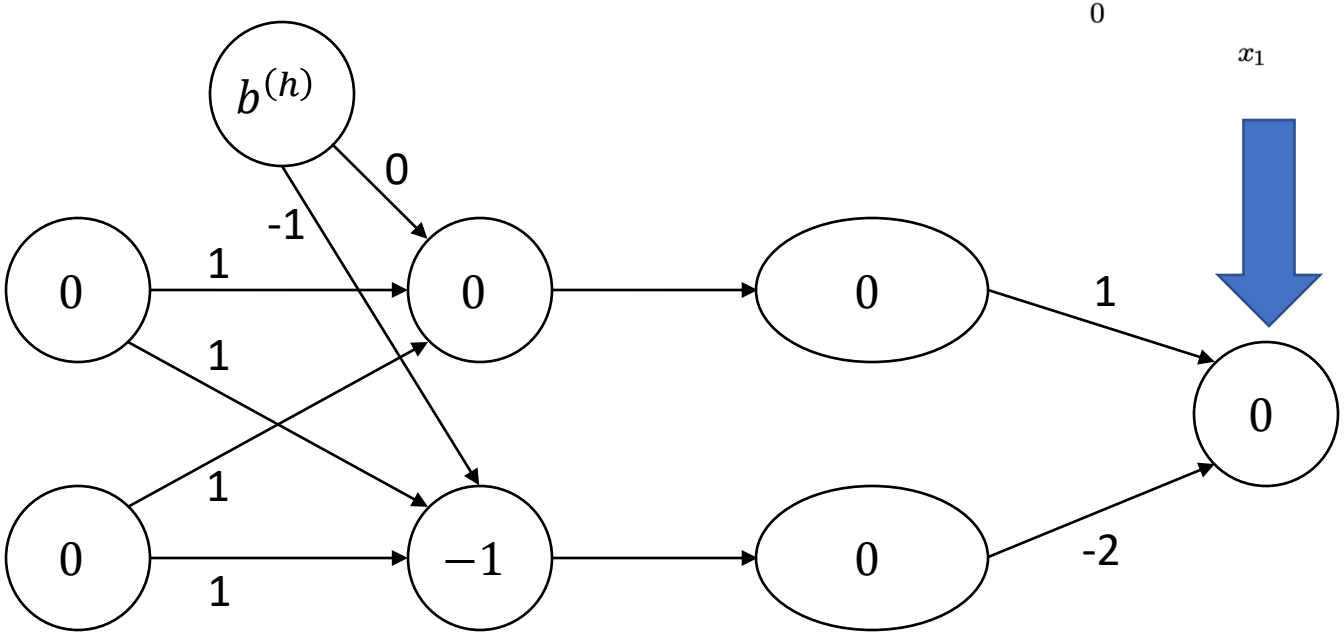




Example: Solving XOR

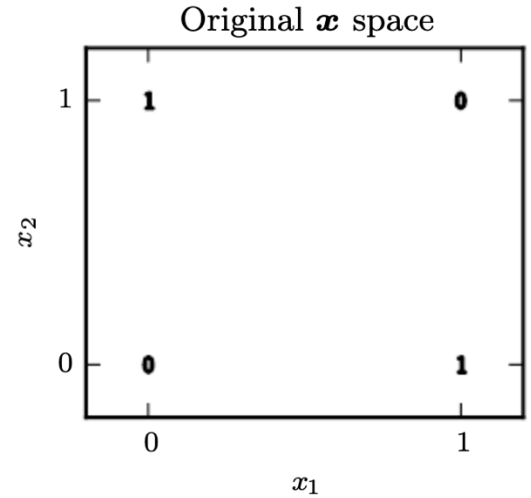
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

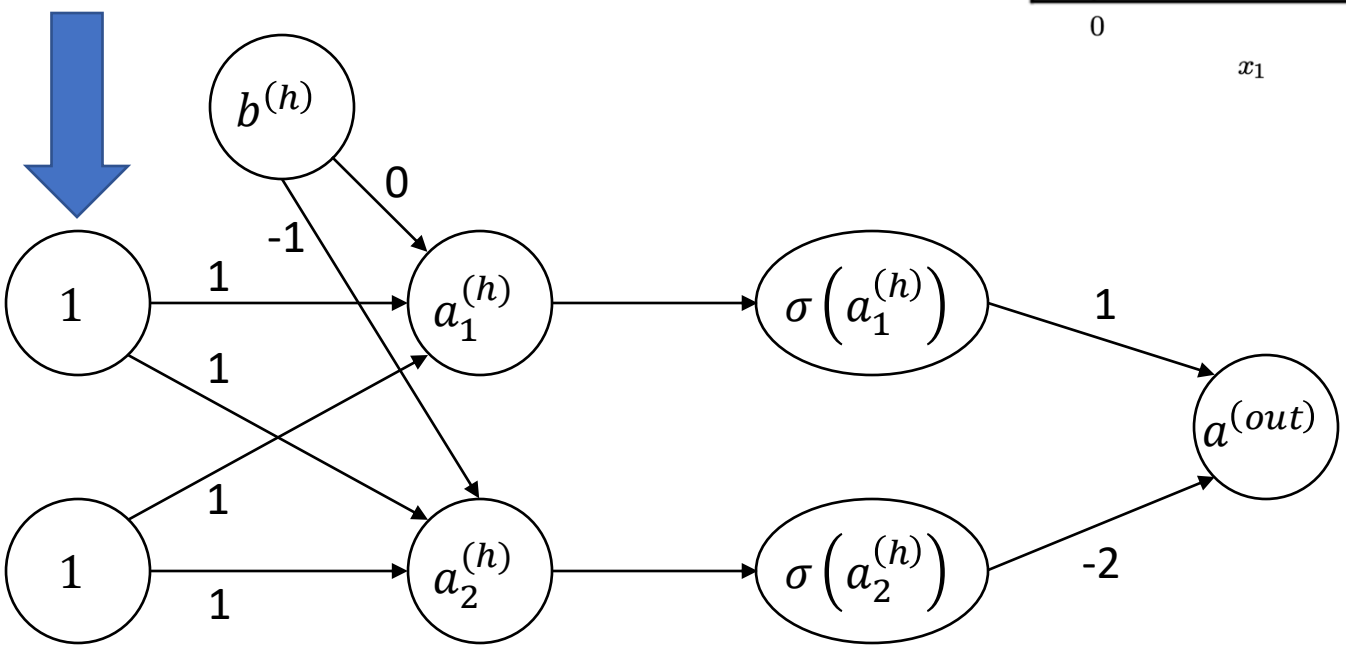




Example: Solving XOR

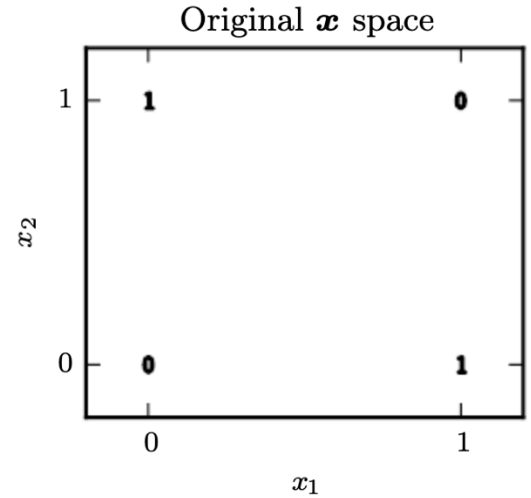
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

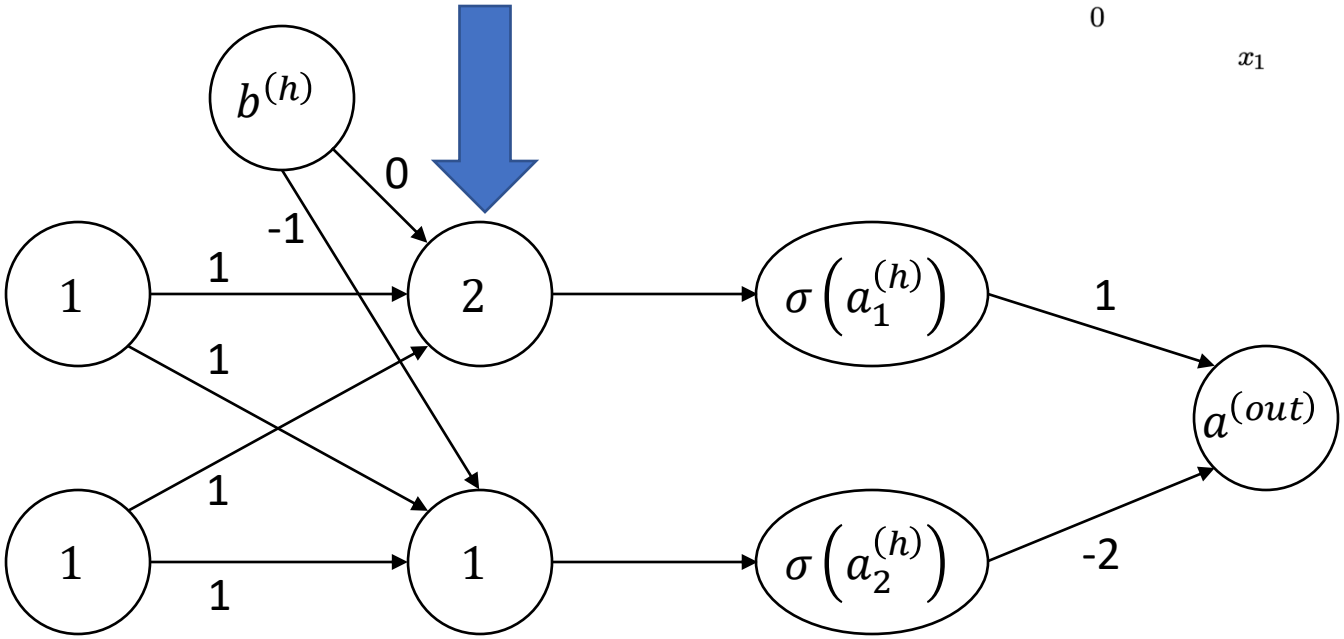




Example: Solving XOR

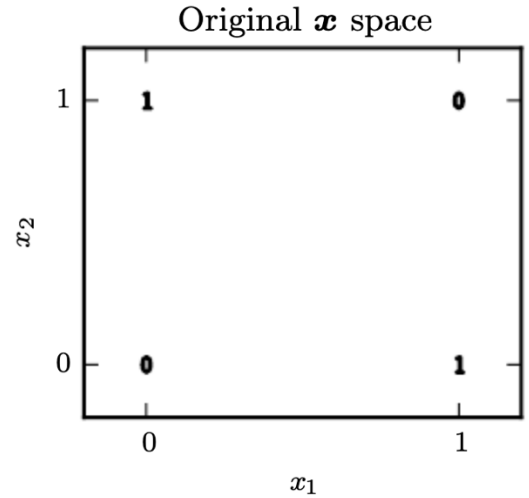
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

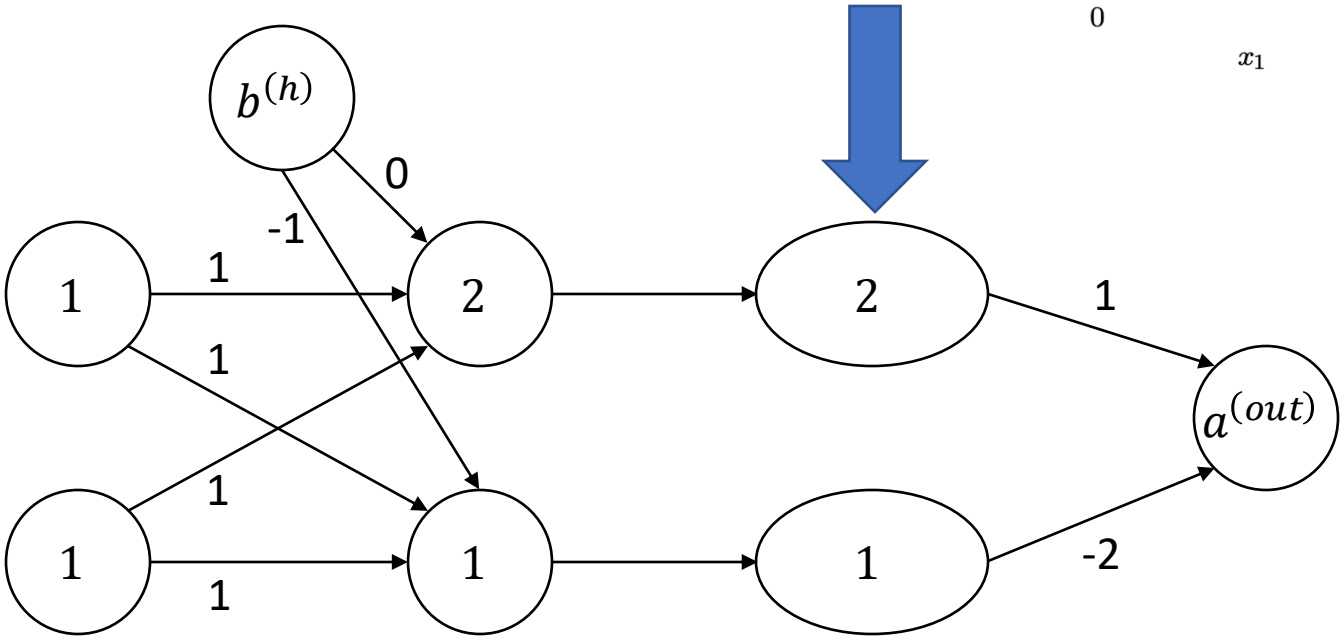




Example: Solving XOR

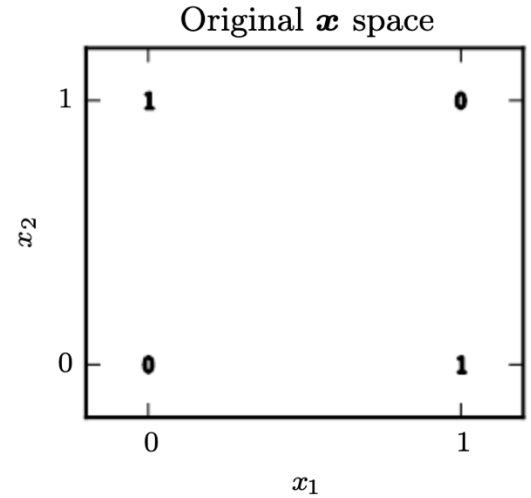
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

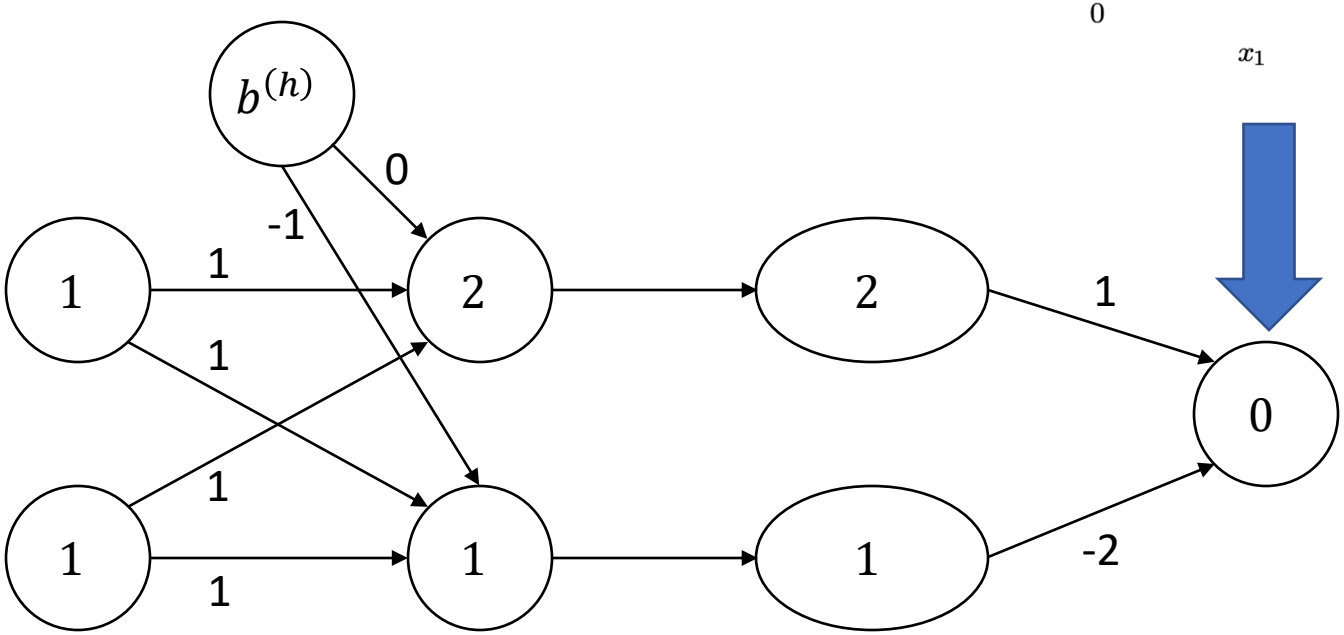




Example: Solving XOR

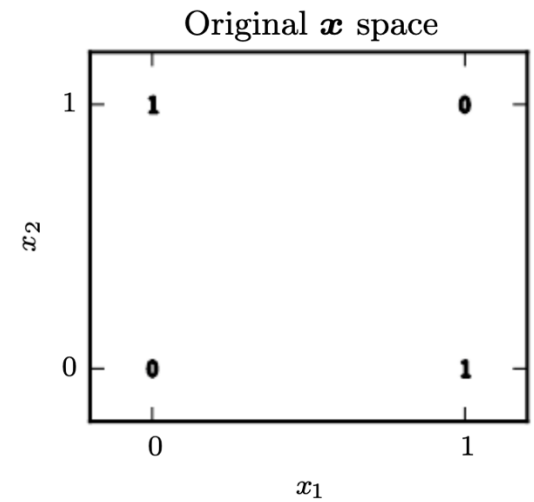
Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$



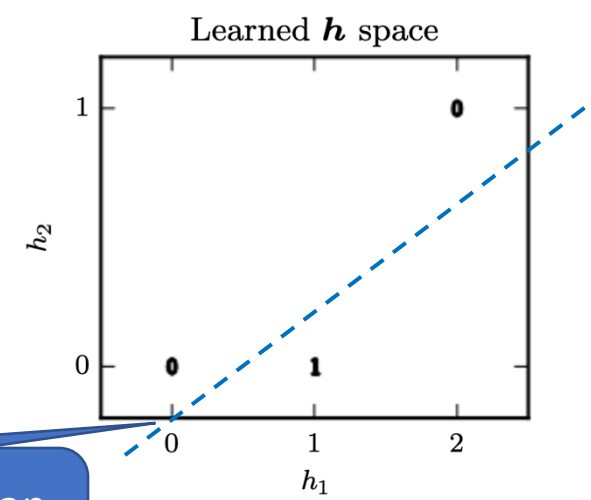


Example: Solving XOR

Inputs $X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ labels $\boldsymbol{y} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$

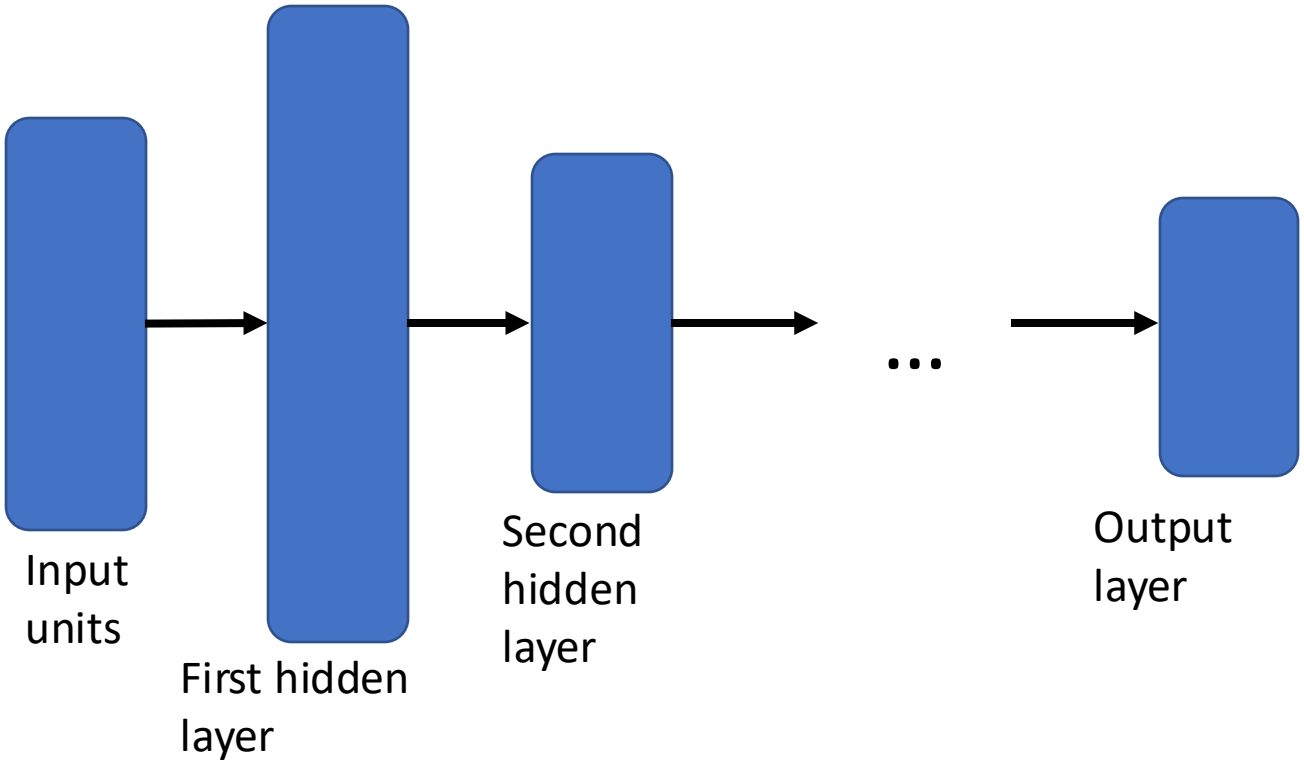


Learned rep (hidden units after activation): $\begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$



Linear decision boundary can classify all correctly

Deeper Models?

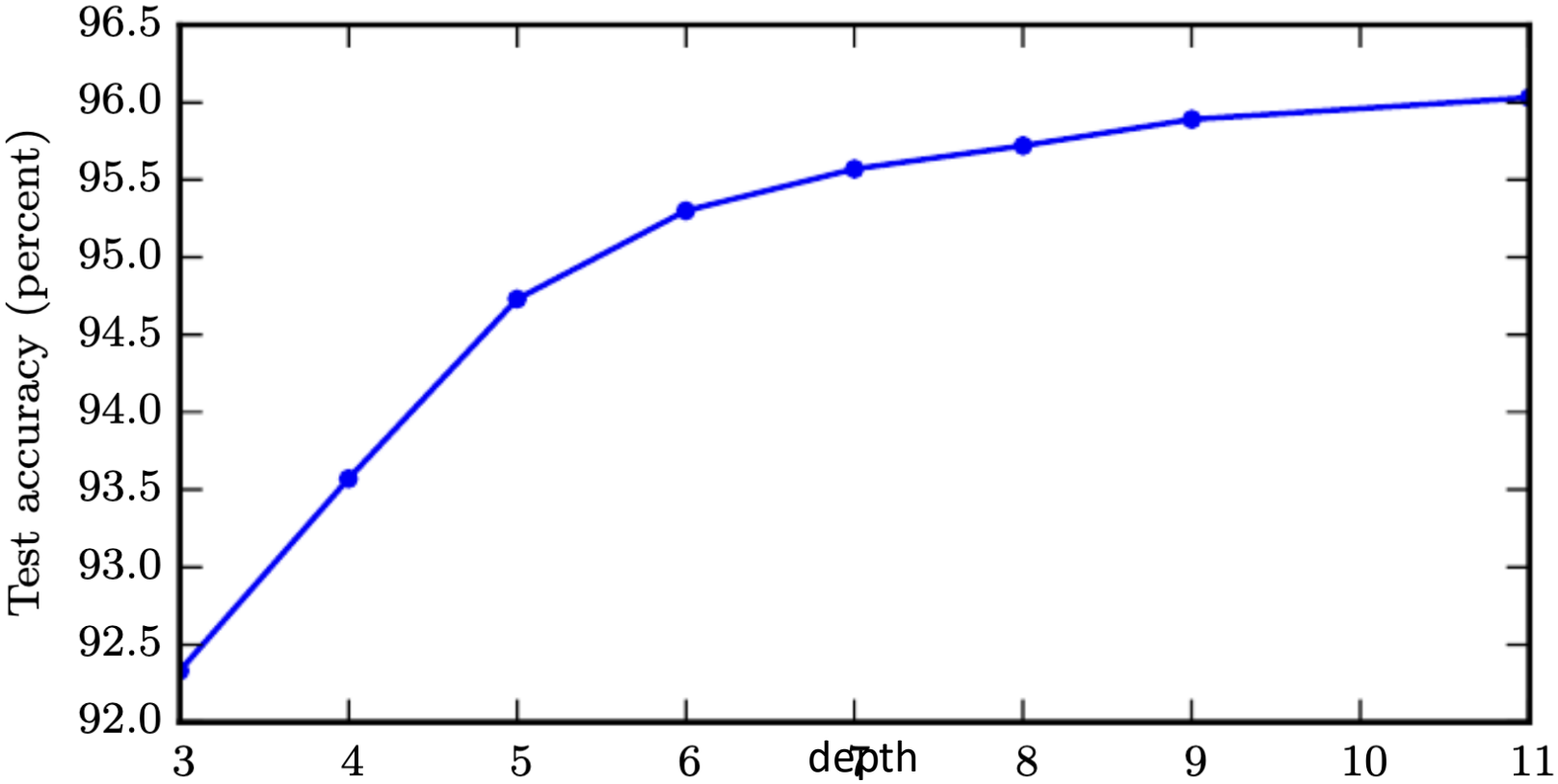


The **depth** of a MLP is the number of hidden layers plus 1 (for the output layer).

A deeper network learns a more hierarchical representation (where the k'th set of hidden units are a transformed representation of that from the k-1'th set of units.

Deeper Models?

GBD 6.6, an ablation study on depth for an image classification task. Holding the total number of parameters constant, some increase in depth leads to better generalization to due learning a more hierarchical representation.



Practical Multilayer Perceptron Architecture Tips

In general, the right architecture is an **experimental** question for validation. But some rules of thumb…

- **Input Size:** Number of features
- **Width (number of hidden units)**: Start with 100-1000, usually 10-100 times the number of features.
- **Nonlinear Activation**: Use ReLU for hidden units by default, sigmoid, tanh, or other as needed
- **Depth**: Start with one or two hidden layers
- **Output and Loss**:
  - **Regression**: Just one unit, no output activation, squared loss
  - **Classification**: Number of classes, softmax normalization, cross entropy loss